TurboQuant v1.5.3

Work Log — Side Buffer Experiment + Double WHT Integration

2026-04-11 ~ 04-12

GPT-OSS 120B · head_dim=64

f16 Side Buffer 35/35

src1 Index Bug Found

3+1 Bit Idea

TBQ4 35/35 Verified

AmesianX

Session Overview

GOAL

Improve TBQP3_3 (3-bit K) math precision.
Achieve f16 K equivalent quality.

RESULTS

Side buffer experiment: 35/35 achieved.
TBQP3_3 original: Korean ✔, math ✘
TBQ4 recommended: 35/35 ✔

KEY DISCOVERY

src1 index mapping — side buffer must use same KV position mapping as set-rows.

IDEA

3+1 bit split — VRAM 3-bit + system RAM 1-bit = 4-bit quality.

f16 Side Buffer Experiment 35/35

> Intercept original K as f16 before quantization into a separate buffer. Attention reads f16 instead of quantized TBQP3_3 data.

- **Phase 1–3:** WHT-domain f16 → raw f16 → include generation tokens
- **Phase 4–5:** Sharpening, *D/D compensation, Q WHT skip — negligible effect
- **Phase 6:** f16 kernel redirect (override K→F16 in fattn.cu) — nearly worked
- **Phase 7:** Direct value comparison → **src1 index mismatch discovered!**
- **Phase 8:** Indexed write applied → **35/35 math bench, multi-turn verified**

Root Cause: src1 Index BUG

set-rows stores K at specific KV cache positions via `src1[i]`.
Side buffer stored sequentially (0,1,2...) → **position mismatch**.

COMPONENT	BEFORE FIX	AFTER FIX
KV cache	src1[0]=5 → position 5	Same
Side buffer	**token 0 → position 0 ✘**	**token 0 → position 5 ✔**

Lesson: Measure, don't guess. Found by directly comparing f16 KV cache values vs side buffer values at the same token position.

3+1 Bit Split Idea

Not Implemented

VRAM 3-bit (non-negotiable) + System RAM 1-bit = 4-bit equivalent quality

MODE	VRAM	RAM	QUALITY	SPEED IMPACT
3+0	3-bit	0	cos 0.945	None
3+1	3-bit	1-bit	cos ~0.990	0.025ms
3+5	3-bit	5-bit	q8_0 equiv	0.125ms
3+13	3-bit	13-bit	f16 equiv	0.31ms

TBQ4 = 35/35 verified → 3+1 bit expected to match. Uses cudaHostAlloc zero-copy.

Math Bench Results 35 Problems

CONFIG	K CACHE	V CACHE	MATH	KOREAN	MULTI-TURN
f16/f16	f16	f16	**35/35**	✔	✔
side buffer	f16 (side)	tbq3_2	**35/35**	✔	✔
tbq4/tbq3	tbq4_2	tbq3_2	**35/35**	✔	✔
tbqp3/tbq3	tbqp3_3	tbq3_2	✘	✔	✔ **(9+ turns)**

Recommended: `–cache-type-k tbq4 –cache-type-v tbq3` → 35/35 math + Korean + multi-turn

v1.5.3 Code Changes

- **common.cpp:** head_dim=64 K → TBQP3_3 mapping (QJL re-enabled)
- **fattn-vec.cuh:** is_cross_head_K=false, is_double_wht_K flag, Q Round 2, /D compensation
- **cpy-utils.cuh:** K encoding double WHT per-head (512-point cross-head removed)
- **fattn.cu:** TBQ_TUNING D=64 full dispatch table
- **CMakeLists.txt:** TBQ_TUNING D=64 instance files registered
- **ggml.h + 24 instances:** Comments: cross-head → double WHT per-head
- **Paper tex/PDF:** Cross-head section replaced with double WHT

Git Status

BRANCH	CONTENT	STATUS
`main`	v1.5.3 + upstream merge	**Pushed**
`experiment/side-buffer-f16`	f16 side buffer 35/35	**Pushed**
`v1.5.3` tag	Release tag	**Published**

CI: CUDA-only workflow (Windows CUDA 12.4/13.1 + Linux Docker CUDA 12.8)
Architectures: sm_70 (V100), sm_80 (A100), sm_86 (3090Ti), sm_89 (4090), sm_90 (H100), sm_100/120 (5090)

Next Steps

TBQP3_3 MATH IMPROVEMENT

- Achieve matrix precision with 3-bit K
- Retune sharpening/softening
- Leverage side buffer findings

3+1 BIT IMPLEMENTATION

- VRAM 3-bit + RAM 1-bit split
- cudaHostAlloc zero-copy
- Modify vec_dot (dual memory read)

MULTI-TURN STABILITY

- Channel tag parser investigation
- LCP caching + side buffer integration

CI / RELEASE

- Verify workflow builds

- Binary release distribution

Speaker notes